



DevOps Internship

Technical Summary Report

Supervisor: Musa Ali Khan
Prepared by: Mubashra Iftikhar
Role: DevOps Intern
Duration: 1 July – 10 August
Submission Date: 6 August 2026

Table of Contents

Learning Track Overview	3
Linux Systems Administration	5
Networking Fundamentals	5
Containerization – Docker	7
Continuous Integration – Jenkins.....	8
Container Orchestration – Kubernetes	10
Centralized Logging – ELK Stack	12
Monitoring & Observability – Prometheus & Grafana.....	13
Version Control & Collaboration – Git & GitHub	15
Capstone Project – CI/CD Pipeline for an AWS-Hosted Service	18
Skills Acquired – Summary	21

1. Purpose

This report summarizes the technical scope and outcomes of the DevOps internship completed at Abacus. It consolidates the tools, concepts, and hands-on implementations covered during the internship — from foundational Linux and networking knowledge through to the design and implementation of a production-style CI/CD pipeline for an AWS-hosted service. Detailed task-level implementation notes, configuration files, and walkthroughs are maintained in the accompanying GitHub documentation repository; this document presents the consolidated engineering summary.

2. Scope of the Internship

The internship followed a progressive, tool-by-tool structure, beginning with operating-system and networking fundamentals and advancing through containerization, CI/CD automation, orchestration, observability, and version control practices. The final phase of the internship consolidated this knowledge into a single capstone project: an end-to-end CI/CD pipeline for an AWS-hosted service, integrating GitHub Actions, container security scanning, Amazon ECR, and a self-managed Kubernetes (K3s) cluster running on EC2.

3. Learning Track Overview

The table below summarizes the technology areas covered during the internship and the core competencies developed in each.

Area	Focus
Linux	Command-line administration, process/network inspection (netstat), service management (systemd)
Networking	IPv4/IPv6 addressing, TCP/UDP transport behavior, HTTP status code semantics
Docker	Image/container lifecycle, networking, volumes, multi-stage builds, image optimization
Jenkins	Declarative/scripted pipelines, credentials management, parametrized and automated build-and-push pipelines
Kubernetes	Cluster architecture, workload types, health probes, service networking, ConfigMaps/Secrets
ELK Stack	Centralized log shipping and indexing, real-time error alerting pipeline

Area	Focus
Prometheus & Grafana	Metrics collection (pull model), exporters, metric types, dashboarding fundamentals
Git & GitHub	Branching strategy, protected branches, GitHub Actions workflow automation
Capstone (AWS)	End-to-end CI/CD pipeline: quality gates, image scanning, ECR, OIDC/IAM auth, K3s on EC2, automated CD with rollback

4. Linux Systems Administration

Linux formed the operating-system foundation for the remainder of the internship, since every tool covered afterward — Docker, Jenkins, Kubernetes nodes, the ELK stack, and the EC2-hosted K3s cluster — runs on Linux hosts. The focus areas were command-line proficiency, runtime/network visibility, and service lifecycle management.

4.1 Command-Line Proficiency

- **Directory Navigation & File Operations** — Creating, moving, copying, and organizing files and directories using core commands such as `cd`, `ls`, `cp`, `mv`, and `rm`.
- **File Inspection & Text Searching** — Viewing and searching file and log contents using commands such as `cat`, `less`, `grep`, and `find`.
- **File Permission and Ownership** — Managing access control at the file and directory level using `chmod` and `chown` within Linux's user/group/others permission model.
- **System and Process Management** — Monitoring and controlling running processes and system resource usage using commands such as `ps`, `top`, and `kill`.
- **netstat** — Inspecting active network connections, listening ports, and routing information to verify that a service is correctly bound to and reachable on its expected port.
- **Service Management – systemd** — Managing long-running services as native OS-level units — enabling, starting, stopping, and inspecting logs for services such as Elasticsearch, Kibana, and Filebeat.

5. Networking Fundamentals

Networking concepts were covered to build the mental model needed to reason about how containers, services, and pipeline stages communicate — both within a single host and across the Docker, Kubernetes.

5.1 IPv4 and IPv6 Addressing

- **IPv4** — 32-bit, dotted-decimal addressing (e.g. 192.168.1.1) providing roughly 4.3 billion unique addresses.
- **IPv6** — 128-bit, hexadecimal addressing designed to resolve IPv4 address exhaustion, offering a substantially larger address space along with simplified auto-configuration.

5.2 Transport Layer — TCP and UDP

The table below summarizes the core operational differences between the two transport-layer protocols covered.

Aspect	TCP	UDP
Connection Model	Connection-oriented — requires a handshake before data transfer	Connectionless — data sent without establishing a session
Reliability	Guaranteed, ordered delivery with error checking and retransmission	Best-effort delivery; no guarantee of order or arrival
Typical Use Cases	HTTP/HTTPS, APIs, file transfer, database connections	DNS lookups, streaming media, VoIP, telemetry data

5.3 HTTP Status Codes

HTTP status codes were covered by class, along with their operational significance.

1xx — Informational

Indicates that the request was received and understood, and that processing is continuing (e.g. 100 Continue).

2xx — Success

Indicates that the request was successfully received, understood, and accepted (e.g. 200 OK, 201 Created, 204 No Content).

3xx — Redirection

Indicates that further action is required to complete the request, typically directing the client to a different resource location (e.g. 301 Moved Permanently, 304 Not Modified).

4xx — Client Error

Indicates that the request contains invalid syntax or otherwise cannot be fulfilled due to a client-side issue (e.g. 400 Bad Request, 401 Unauthorized, 404 Not Found).

5xx — Server Error

Indicates that the server failed to fulfill a valid request due to an internal fault (e.g. 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable).

6. Containerization — Docker

Docker was the entry point into the containerization and CI/CD toolchain used throughout the rest of the internship. Coverage spanned the Docker architecture (client, daemon, registry, and objects), image and container lifecycle management, networking drivers, and persistent storage.

6.1 Core Concepts

- **Docker Client & Daemon** — The CLI (docker) communicates with the background daemon (dockerd) over a REST API; the daemon manages images, containers, networks, and volumes.
- **Dockerfile** — A text manifest of instructions (FROM, WORKDIR, COPY, RUN, CMD) used to build an image.
- **Docker Image** — A read-only, layered blueprint used to instantiate containers.
- **Docker Container** — An isolated, runnable process instance created from an image.
- **Networking Drivers:**
 - **bridge (default)** — Private internal host network enabling inter-container communication.
 - **host** — Shares the host's network interface directly, with no isolation.
 - **none** — Disables networking entirely.
 - **overlay** — Connects containers across multiple Docker hosts.
- **Storage Management:**
 - **Docker Volumes** — Managed by Docker under `/var/lib/docker/volumes/`; recommended for persistent production data.
 - **Bind Mounts** — Map a host directory path directly into a container; suited to live development workflows.

6.2 Image Optimization Techniques

- **Multi-Stage Builds** — Uses multiple FROM statements to separate heavy build tooling from the final, lean production image.
- **Minimal Base Images** — Uses alpine or slim variants (e.g. python:3.11-alpine), reducing image size from roughly 1 GB to under 100 MB.
- **Layer Caching** — Combines related RUN commands (e.g. RUN apt-get update && apt-get install -y ...) and copies package manifests before source code to maximize Docker's build-cache reuse.
- **.dockerignore** — Excludes build artifacts, .git, and node_modules from the build context.

6.3 Applied Project

A multi-container, three-tier application was deployed using Docker Compose, orchestrating an NGINX frontend, a Node.js/Express API backend, and a PostgreSQL database as independent services within a single stack.

- **Service Orchestration** — Configured docker-compose.yml to orchestrate three services: an NGINX frontend, a Node.js/Express API backend, and a PostgreSQL database.
- **Persistent Storage** — Configured a named persistent volume for the PostgreSQL database, preserving data across container restarts.

7. Continuous Integration — Jenkins

Jenkins is a tool that automatically builds, tests, and deploys code whenever a change is made, removing the need to repeat these steps by hand. Coverage included its core components, pipeline configuration, and secure handling of credentials.

7.1 Architecture & Components

- **Controller (Master)** — Manages the UI, pipeline configurations, and plugins, and dispatches jobs to agents.
- **Agent (Node)** — A machine or container that executes the build steps dispatched by the controller.
- **Executor** — A slot on an agent that runs a single build; multiple executors allow concurrent job execution.
- **Jenkinsfile** — A text file defining pipeline steps as code (pipeline-as-code).

7.2 Pipeline Paradigms & Configurations

- **Declarative vs Scripted Pipeline:**
 - **Declarative** — Structured, simplified syntax (pipeline { ... }); recommended for standard CI/CD.
 - **Scripted** — Written in Groovy (node { ... }); offers full programming flexibility such as loops and complex conditionals.
- **Parametrized Pipeline:**
 - **string** — Accepts free-form text input from the user.
 - **choice** — Presents a dropdown list of predefined options.
 - **booleanParam** — A simple true/false checkbox.

- **Secrets Security** — Integrated with Jenkins' Credentials Manager; secrets are referenced by ID and masked in build logs.
- **Post Action Conditions:**
 - **always** — Runs regardless of the build outcome.
 - **success** — Runs only if the build succeeds.
 - **failure** — Runs only if the build fails.
 - **changed** — Runs if the build status differs from the previous build.
 - **fixed** — Runs if the current build succeeded after a previous failure.
 - **cleanup** — Runs last, to clean up workspace files.

7.3 Applied Tasks

- Deployed Jenkins via Docker and configured it as a working automation server.
- Implemented an automated build-and-push pipeline that builds a Docker image on code changes and pushes it to a registry.
- Implemented a parametrized pipeline with Slack notifications for build status reporting.

8. Container Orchestration — Kubernetes

Kubernetes is a container orchestration platform used to automate deployment, scaling, self-healing, load balancing, and rolling updates across cluster nodes — addressing exactly what becomes unmanageable with plain Docker at scale.

8.1 Architecture and Health Model

- **Control Plane (Master Node):**
 - **kube-apiserver** — Exposes the cluster API; the central control hub for kubectl.
 - **etcd** — Distributed key-value store holding the complete cluster state.
 - **kube-scheduler** — Assigns newly created pods to the optimal worker node based on available resources.
 - **kube-controller-manager** — Runs background control loops that enforce the cluster's desired state.
- **Worker Node:**
 - **kubelet** — The primary agent on a worker node, ensuring containers are running and healthy.
 - **kube-proxy** — Maintains network routing rules for Service abstractions.
 - **Pod** — The smallest deployable unit, containing one or more co-located containers.
- **Health Probes:**
 - **StartupProbe** — Checks whether the application's initial startup is complete; disables the liveness and readiness probes until it passes.
 - **LivenessProbe** — Checks whether the container is running and healthy; on failure, kubelet restarts the container.
 - **ReadinessProbe** — Checks whether the container is ready to accept traffic; on failure, the pod is removed from Service endpoints.
- **Pod Lifecycle Phases** — Pending → Running → Succeeded / Failed.

8.2 Workload and Service Types

Workload	Use Case
Deployment	Stateless services (web servers, APIs) — supports rolling updates.
StatefulSet	Stateful services requiring stable identity and persistent storage (e.g. databases)

Workload	Use Case
DaemonSet	One pod per node – log collectors, monitoring/networking agents

Service exposure was tested across ClusterIP, NodePort, and LoadBalancer types on a Minikube cluster, validating reachability from inside a pod, from the host OS, and from an external client for each type.

Service Type	Description
ClusterIP (default)	Exposes the service on an internal cluster IP, for internal-only communication
NodePort	Opens a static port (30000–32767) on worker node IPs for external, typically development-time, access
LoadBalancer	Provisions an external cloud load balancer that routes traffic to the service
ExternalIP	Routes traffic arriving at a specified external IP directly to the service

8.3 Applied Project

The three-tier Student Registration application (Section 6.3) was migrated to Kubernetes: the Node.js backend API was deployed as a Deployment with 3 replicas, and the MySQL database was deployed as a StatefulSet backed by a PersistentVolumeClaim (PVC) for stable, persistent storage. The frontend was exposed through a NodePort Service, with internal API calls routed through a ClusterIP Service.

9. Centralized Logging – ELK Stack

A self-hosted logging pipeline was built using Elasticsearch, Kibana, and Filebeat, run as native systemd services on a single Ubuntu host, then extended into a real-time HTTP error alerting pipeline.

9.1 Components

- **Elasticsearch (port 9200)** — The datastore and search engine that indexes and stores log data for querying.
- **Kibana (port 5601)** — The web-based GUI used to visualize, search, and explore data indexed in Elasticsearch.
- **Filebeat** — A lightweight log shipper that reads log files on a host and forwards them to Elasticsearch for indexing.

9.2 Real-Time Error Alerting

Extended the base stack with a Dockerized demo application that generates genuine HTTP errors, shipped through a dedicated Filebeat instance into a separate index, with a purpose-built Kibana dashboard and alerting configured to notify by email when error rates spike or the service goes silent — moving the stack from passive log storage to active operational alerting.

10. Monitoring & Observability — Prometheus & Grafana

Prometheus and Grafana coverage focused on metrics-based observability as a complement to the ELK stack's log-based approach — covering how metrics are collected, stored, queried, and visualized, and how alerting is wired on top of them.

10.1 Prometheus & Grafana

- **Prometheus (port 9090)** — A time-series monitoring system that pulls (scrapes) metrics from targets over HTTP, stores them in its own time-series database, and answers queries via PromQL.
- **Grafana (port 3000)** — A visualization and alerting layer that connects to Prometheus as a data source, turning scraped metrics into dashboards and configurable alert rules.
- **Node Exporter (port 9100)** — A lightweight agent that runs on a host and exposes hardware and OS-level metrics (CPU, memory, disk, filesystem, network)

10.2 Architecture

- **Data Retrieval Workers** — Pull metric data from targets over HTTP at a configured scrape interval.
- **Time Series Database (TSDB)** — Stores all scraped metric data over time.
- **HTTP Server** — Accepts PromQL queries from Prometheus's own Web UI or from Grafana.

Grafana sits outside Prometheus's internal architecture — it queries the same HTTP Server that Prometheus's own UI uses, without ever touching its storage or scraping logic directly.

10.3 Targets & Metrics

- **Target** — Anything Prometheus monitors: an application, a server, or a backend service.
- **Metric** — A specific unit measured on a target, such as CPU usage, request count, or request duration.
- **Application metrics** — Custom, business-level metrics instrumented directly into an application (e.g. login attempts, login failures by reason, request latency, active sessions).
- **Server / system metrics** — Host-level metrics such as CPU, memory, disk, and network usage, typically exposed via an exporter (e.g. Node Exporter) rather than the application itself.

10.4 Metric Types

Metric Type	Behavior	Example
Counter	Monotonically increasing	Total HTTP requests
Gauge	Value moves up or down	CPU usage, active sessions
Histogram	Distribution of observed values	Request latency buckets

10.5 Collection Model

Covered Prometheus's pull-based scraping model, the role of exporters (e.g. `node_exporter`) for services that don't natively expose metrics in Prometheus format, and the Pushgateway pattern used for short-lived jobs and push-only sources that don't fit the default pull model.

10.6 Applied Project

A self-hosted monitoring and alerting stack was built end-to-end on a Linux VM: Prometheus and Node Exporter were installed as `systemd` services, and a custom Flask login application was instrumented using the `prometheus_client` library to expose application-level metrics (login attempts, successes, failures by reason, request latency, and active sessions). Grafana was connected to Prometheus as a data source to build dashboards for both the custom application and full system metrics, and a CPU-usage alert rule was configured in Grafana's native alerting engine, with email notifications delivered through Gmail SMTP when the threshold is breached.

11. Version Control & Collaboration – Git & GitHub

This module covered Git and GitHub as the version-control and automation backbone for every project built during the internship, culminating in the branching strategy and GitHub Actions workflows implemented directly in the capstone pipeline (Section 12).

This module covered Git and GitHub as the version-control and automation backbone for every project built during the internship, culminating in the branching strategy and GitHub Actions workflows implemented directly in the capstone pipeline (Section 12).

11.1 Git Commands

Core Git commands and merge strategies used to keep local work in sync with the remote and to integrate changes cleanly across branches.

git fetch vs. git pull

git fetch	git pull
Downloads the latest commits/refs from the remote without touching local files	Downloads the latest commits and immediately merges (or rebases) them into the current branch
Left untouched – nothing changes until you merge manually	Updated right away with the incoming changes
To check what is new before deciding how to integrate it	To bring a local branch directly up to date with its upstream

Merge vs. Rebase

Merge	Rebase
Preserves full history, including a merge commit	Rewrites commit history onto a new base – linear, no merge commit
Non-linear; shows exactly when branches diverged and joined	Linear; looks as if work was done sequentially on top of the target branch
Merging feature branches into dev, uat, or prod (shared, protected branches)	Cleaning up a local feature branch before opening a pull request
Low Risk— safe on shared/public branches	Higher risk – should never be done on a branch others are already using, since it rewrites history

git stash – temporarily shelves uncommitted changes so the working directory is clean, e.g. to switch branches or pull updates without committing half-finished work.

git cherry-pick — applies one specific commit from one branch onto another without merging the entire branch. Used to port a hotfix commit into dev/uat without pulling in all of prod's history.

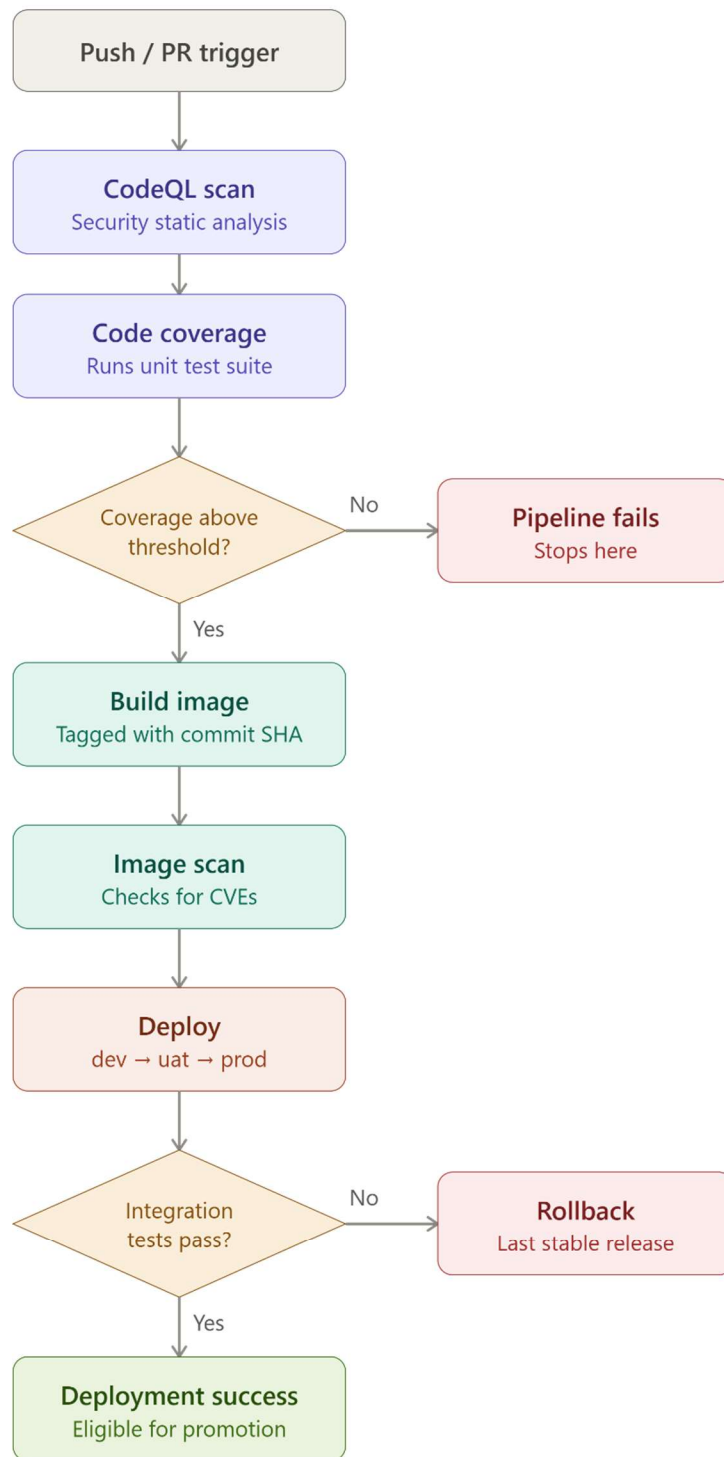
11.2 Branching Strategy & GitHub Actions Workflow

The branching model and CI/CD automation that together govern how code moves from a feature branch to a live deployment.

- **Protected branches** — prod, uat, and dev only accept changes via pull request, gated by required CI checks.
- **Feature branches cut from prod** — feature/fix/hotfix branches always branch off prod, so they start from a known-good, deployable state.
- **Promotion flow** — validated changes move feature → dev → uat → prod, with review, sync with prod and CI gating each step.
- **Hotfixes** — cut from prod, merged straight back to prod, then back-merged into uat and dev so branches do not drift.

11.3 GitHub Actions Workflows

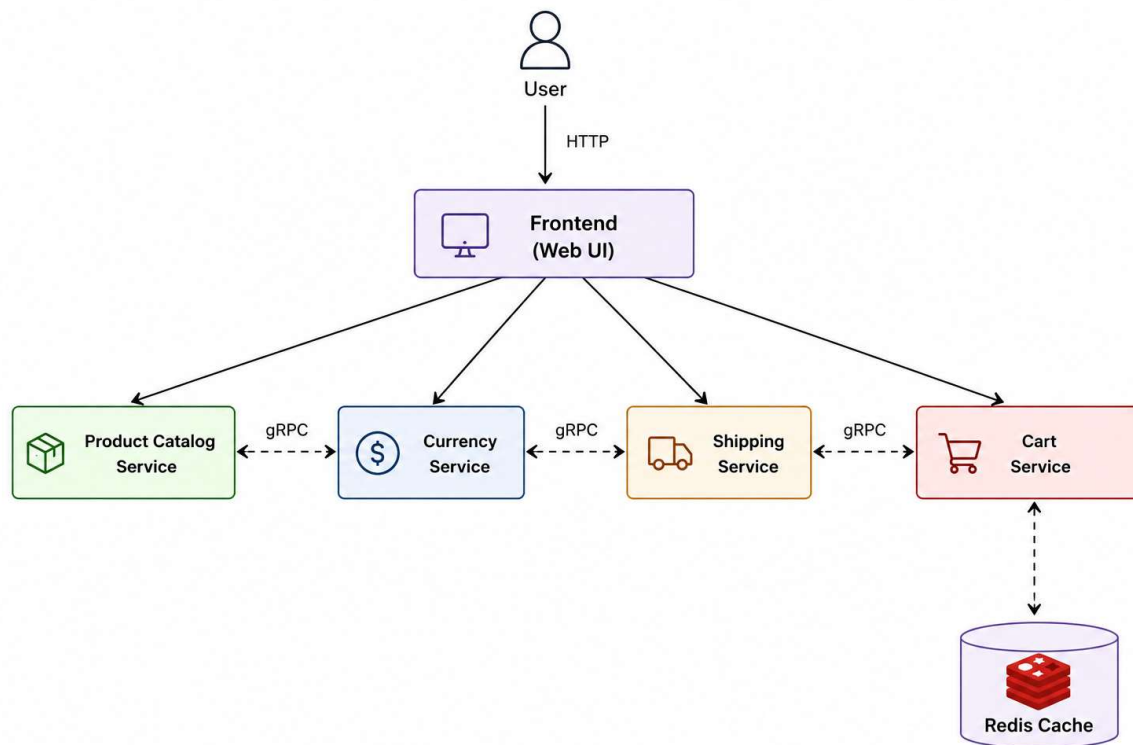
Built GitHub Actions workflows to automate build, test, and deployment steps in response to repository events (push and pull-request triggers).



12. Capstone Project – CI/CD Pipeline for an AWS-Hosted Service

The internship's capstone project consolidated every preceding module into a single, engineered CI/CD pipeline for a service hosted on AWS. The pipeline is triggered from GitHub, enforces code-quality and security gates before an image is built, publishes the image to a private registry, and deploys onto a self-managed Kubernetes (k3s) cluster running on EC2 – with an automated integration-testing gate controlling whether a deployment is promoted or rolled back.

12.1 Architecture Overview



The pipeline is organized into four logical stages. Each stage is a gate for the next — a failure at any stage halts the pipeline before it can reach production.

Stage	What Happens
1. Source & CI Gates	Branching strategy enforced on PRs; lint, CodeQL static analysis, and code coverage checks run before merge
2. Build & Scan	Docker image built from the merged commit; image vulnerability scan run against the built artifact
3. Publish	Authenticate via OIDC/IAM; push the scanned image to Amazon ECR
4. Deploy & Verify (CD)	Deploy to K3s on EC2; run integration tests; promote on pass, automatically revert to the last stable version on failure

12.2 CI Stage — Code Quality & Build Gates

Pull requests go through three checks before merging: linting for code style, CodeQL for security issues, and test coverage — meant to catch problems early, before an image is even built.

12.3 Image Build, Security Scanning & Registry Publish

Once merged, the pipeline builds the Docker image, runs a Trivy scan for known vulnerabilities, and pushes it to ECR using short-lived OIDC credentials rather than a stored AWS key.

12.4 Deployment Target — K3s on EC2

The app runs on K3s, a lightweight Kubernetes distribution, on a single EC2 instance — a way to work with real Kubernetes concepts without the cost of a managed control plane.

12.5 CD Stage — Automated Integration Testing & Rollback

The final stage applies a deploy-verify-decide pattern rather than a one-way deployment:

- **Deploy** — the newly published image is rolled out to the K3s cluster.
- **Verify** — an automated integration test suite is executed against the newly deployed version to confirm the service behaves correctly end-to-end, not just that the container started.

- **Decide** — on a test failure, the pipeline automatically reverts the deployment to the most recent known-stable version, containing the failure without requiring manual intervention; on a pass, the new version remains live as the current stable release.



13. Skills Acquired – Summary

- Linux administration and diagnostics – command-line proficiency, network/process inspection, systemd service management.
- Core networking – IPv4/IPv6 addressing, TCP/UDP transport behavior, HTTP status code interpretation.
- Containerization – Docker image/container lifecycle, networking, volumes, and image optimization.
- CI/CD automation – Jenkins pipeline-as-code and credential management; GitHub Actions workflow design.
- Orchestration – Kubernetes architecture, workloads, health probes, and service networking; hands-on K3s cluster operation.
- Observability – centralized logging and alerting with the ELK stack; metrics-based monitoring with Prometheus and Grafana.
- Version control governance – branching strategy, protected branches, and PR-gated quality checks.
- End-to-end pipeline engineering – quality gates (lint, CodeQL, coverage), image security scanning, OIDC/IAM keyless authentication, container registry publishing, and an automated integration-test-gated CD process with rollback.